

Prompt

Je veux implémenter un système où chaque utilisateur peut configurer sa propre boîte email SMTP pour envoyer les emails de prospection depuis son adresse personnelle (Gmail, Outlook, etc.) au lieu d'une adresse globale.

Contexte du projet

- Next.js + TypeScript + Prisma (SQLite) + Nodemailer
- Auth custom avec JWT (pas NextAuth)
- Le user connecté est identifiable via le token JWT dans les cookies (`authToken`)
- Le fichier d'envoi actuel est `app/api/emails/send/route.ts` – il utilise `process.env.SMTP\_\*` global, il faut le modifier pour utiliser les credentials du user connecté
- Le schéma Prisma est dans `prisma/schema.prisma`

Ce qu'il faut faire

1. Modifier `prisma/schema.prisma`

Ajouter ces champs au model `User` :

```
``prisma
smtpEmail      String?
smtpAppPass    String? // stocké chiffré
smtpHost       String? // ex: smtp.gmail.com
smtpPort       Int?    // ex: 587
smtpConfigured Boolean @default(false)
```

Puis générer la migration : `npx prisma migrate dev --name add_smtp_config`

2. Créer `lib/crypto.ts`

Utilitaire pour chiffrer/déchiffrer l'App Password avec AES-256-CBC en utilisant `process.env.ENCRYPTION_KEY` (32 caractères). Exporter deux fonctions : `encrypt(text: string): string` et `decrypt(encrypted: string): string`.

3. Créer `app/api/user/smtp-config/route.ts`

- **GET** : retourner la config SMTP du user connecté (smtpEmail, smtpHost, smtpPort, smtpConfigured) SANS renvoyer le mot de passe
- **POST** : sauvegarder les credentials SMTP du user (chiffrer l'App Password avec `lib/crypto.ts` avant de le stocker), mettre `smtpConfigured: true`
- **DELETE** : supprimer la config SMTP du user (remettre tous les champs à null, `smtpConfigured: false`)
- Ajouter une route **POST** `app/api/user/smtp-config/test/route.ts` : tester la connexion SMTP avec les credentials fournis sans les sauvegarder (utiliser `transporter.verify()`)

4. Modifier `app/api/emails/send/route.ts`

- Extraire le `userId` depuis le JWT dans le cookie `authToken`
- Récupérer les credentials SMTP du user depuis la base Prisma

- Si `smtpConfigured` est false → retourner une erreur 400 claire : "Veuillez configurer votre adresse email dans les paramètres avant d'envoyer des emails"
- Déchiffrer l'App Password avec `lib/crypto.ts`
- Créer le transporter Nodemailer dynamiquement avec les credentials du user (ne plus utiliser `process.env.SMTP_*`)
- Le from doit utiliser l'email du user : `"${fromName}" <${user.smtpEmail}>`

5. Créer la page `app/dashboard/settings/email/page.tsx`

Une page de paramètres avec :

- Un formulaire avec les champs : Email, App Password (input type password), Fournisseur (select : Gmail / Outlook / Yahoo / Autre)
- Quand on choisit un fournisseur, pré-remplir automatiquement le SMTP host et port :
 - Gmail → `smtp.gmail.com` : 587
 - Outlook → `smtp.office365.com` : 587
 - Yahoo → `smtp.mail.yahoo.com` : 587
 - Autre → champs libres
- Bouton "Tester la connexion" qui appelle la route de test avant de sauvegarder
- Bouton "Sauvegarder"
- Bouton "Supprimer la configuration"
- Afficher l'état actuel (configuré  ou non configuré )

6. Créer un composant `app/dashboard/settings/email/GmailGuide.tsx`

Un guide visuel step-by-step pour aider le client à obtenir son App Password Gmail :

- Étape 1 : Aller sur `myaccount.google.com` → Sécurité
- Étape 2 : Activer la validation en 2 étapes (avec lien direct)
- Étape 3 : Chercher "Mots de passe des applications" (avec lien direct : <https://myaccount.google.com/apppasswords>)
- Étape 4 : Dans "Sélectionner une application" → choisir "Autre" → nommer "Hatmada"
- Étape 5 : Copier le code à 16 caractères généré
- Étape 6 : Le coller dans le champ "App Password" ci-dessus Afficher ce guide uniquement si le fournisseur sélectionné est Gmail. Afficher un guide similaire simplifié pour Outlook (lien vers <https://account.microsoft.com/security>).

7. Mettre à jour `.env.local.example`

Ajouter la variable : `ENCRYPTION_KEY=your_32_character_encryption_key`

Contraintes

- Utiliser le middleware JWT existant pour authentifier toutes les nouvelles routes API (vérifier le cookie authToken ou le header Authorization: Bearer)
- Ne jamais renvoyer l'App Password déchiffré au client
- Le transporter Nodemailer doit être créé à la demande (pas en singleton global) puisque chaque user a ses propres credentials
- Garder le style cohérent avec le reste du dashboard (Tailwind CSS, thème sombre)